Sequence Models for Linux Task Scheduling: A TCN Addition to CFS Prediction

Matthew Marcotullio
University of California, Davis
Computer Science
Davis, California, USA

Michael Cho
University of California, Davis
Computer Science
Davis, California, USA

Mansheel Agarwal
University of California, Davis
Computer Science
Davis, California, USA

Madhav Rajesh
University of California, Davis
Computer Science
Davis, California, USA

Abstract

Process scheduling is one of the most performance-critical components of an operating system. The Linux Completely Fair Scheduler (CFS) makes decisions purely from instantaneous kernel state, with no memory of historical execution patterns. KernelOracle [5] demonstrated that scheduling sequences are learnable from traces using an LSTM network, but the model was trained on only a single workload and inference latency was far too high for real-time use. The goal of this project is to address both, building off of this repository [4]. We build a new data collection pipeline using Linux `ftrace` to capture scheduling traces across diverse workloads, including CPU-bound, I/O-mixed, and scheduler-stress scenarios. We then replace KernalOracle's Long Short Term Memory (LSTM) approach with a Temporal Convolutional Network (TCN), an architecture that processes sequences in parallel and maintains lower inference latency. After five epoch of training, our TCN achieves accuracy and inference latency which outperforms the LSTM baseline. Our results show that Linux scheduling behavior is highly predictable across diverse workloads, and that a TCN is a more suitable model architecture than an LSTM for this task. Our implementation is in this following link: https://github.com/mmarcotullio/KernelOracle

1 Introduction

Process scheduling determines which task runs on a CPU at any given instant. In Linux, the Completely Fair Scheduler (CFS) promotes fairness among tasks, but ignores how tasks have behaved in the past. In other words, decisions are made only from the instantaneous state of the run queue. This limitation prevents the scheduler from making intelligent decisions. If the scheduler knew in advance which task it was likely to schedule next, it could take preemptive actions.

KernelOracle [5] asks an important question: can a deep learning model learn to predict what the Linux scheduler will do next? It collected scheduling traces using the `perf sched` tool, and trained an LSTM to predict the next-scheduled task. The results were encouraging However, we identified three significant shortcomings within their methodology:

(1) **Regression formulation.** The LSTM was trained with MSE loss over continuous feature vectors, treating scheduling as a regression problem. We view this to not be optimal because scheduling is a discrete choice from a set of processes. No classification metrics were reported.
(2) **Single workload.** The model only used a single trace. The processor remained busy by starting an nginx web server, and sending dummy requests using a tool called Apache Bench (ab). There was no evaluation on different workload types, leaving its generalization ability unknown.
(3) **High inference latency.** The paper acknowledged that their inference latency is "orders of magnitude" higher than the scheduler's own decision time, but did not provide measurements. The sequential aspect of LSTMs makes low-latency inference difficult to achieve.

We build on KernelOracle with the following contributions:

(1) A new **data collection pipeline** using Linux `ftrace` that captures scheduling traces across 12 diverse workloads in 5 categories to produce a more robust dataset.
(2) A **TCN next-PID classifier** that treats scheduling prediction as a multi-class classification problem and achieves high accuracy on both seen and unseen workloads.
(3) A **latency measurement** showing that TCN inference (1.393 ms/sample on CPU) is more practical compared to an LSTM for real-time use cases.

2 Background

The Completely Fair Scheduler "basically models an 'ideal, precise multi-tasking CPU' on real hardware" [6]. Since only a single task can run at once on real hardware, the concept

of "virtual runtime" is implemented. The *vruntime* of a task is its actual runtime normalized to the total number of running tasks, and is tracked per-task by CFS. CFS's task picking logic always tries to run the task with the smallest vruntime value. This way, CFS tries to split up CPU time between runnable tasks as close to ideal multitasking hardware as possible.

Kahu [5] collected scheduling data from an Ubuntu 18.04 VM (Linux 4.15, single CPU) running nginx under Apache Bench load. Data was captured using `perf sched map` and preprocessed into a multivariate time series: one-hot encoded task names (28 unique) concatenated with min-max normalized inter-arrival times. A 2-layer stacked LSTM was trained using LBFGS with MSE loss for 30 epochs. Test loss stabilized around 0.015 MSE. Results were presented as waveform plots comparing predicted and ground-truth scheduling sequences. No top-1 accuracy or inference latency was measured.

3 Design and Implementation

Our system consists of three components: a data collection pipeline for generating scheduler trace data, a data preprocessing pipeline, and the LSTM baseline and our TCN model.

3.1 Data Collection

Motivation. The original KernelOracle pipeline used `perf sched map` on a single-CPU VM running one workload. The resulting dataset captured only the nginx/ab scheduling pattern and was limited to roughly 17,000 events. Generalization to other workloads was never evaluated.

ftrace-based collection. We wrote `collect_traces.py`, a Python script that configures Linux `ftrace` directly through the `tracefs` filesystem (the same kernel mechanism used by Google SchedViz [3]). Rather than the simplified `perf sched map` output, we enable four raw kernel tracepoints:

- `sched_switch`: a task is switched onto a CPU
- `sched_wakeup`: a sleeping task becomes runnable
- `sched_wakeup_new`: a newly created task is woken
- `sched_migrate_task`: a task moves between CPUs

However, only `sched_switch` events are parsed into the training CSV. This provides all the necessary information for our current implementation, and contains the following: previous task running, `prev_state` (the reason why it left), and which task was chosen next.

Workloads. We defined 12 workloads across 5 categories, summarized in Table 2. Each workload was run 3 times for approximately 10 seconds per run, resulting in 36 raw trace files. The workload diversity is a large improvement over the baseline: the model must learn from CPU-bound, I/O-mixed, and scheduler-stress patterns simultaneously.

Category	Workloads
`hackbench`	Pipe/socket message-passing
`sysbench_cpu`	Prime computation
`sysbench_threads`	Yield-heavy threads
`sysbench_memory`	Memory access patterns
`cpu_bound`	Python math loops
`io_mixed`	Mixed dd I/O + CPU burn

Table 1: Workload categories and descriptions.

Data format. Each parsed `sched_switch` event becomes one row in the CSV with fields: `timestamp`, `time_diff`, `task_name`, `pid`, `cpu`, `prev_state`, `prev_comm`, `workload_type`, `trace_id`. The `prev_state` field captures *why* the previous task left the CPU (i.e. `R` = preempted, `S` = voluntary sleep, `D` = I/O wait), which is a meaningful signal for learning scheduling behavior.

Train/test split. Splitting is performed at the trace level within each category, not at the event level, as random event-level splitting would leak temporal context across sets. One entire category is randomly selected as the unseen holdout; every trace from that category (all 3 runs) goes into an unseen test set, and the model never sees a single event from that workload during training. The remaining 5 categories are split by trace file at an 80/20 ratio: traces are shuffled per category, with 80% going to train and the rest to the seen test set. Testing on seen and unseen data raises the questions as to whether the model can generalize to familiar and unfamiliar workloads. We used `sysbench_threads` as the unseen holdout. Initial trace analysis confirms a variance in predictability: the unseen set contains background desktop applications falling into repetitive, deterministic context-switch loops that sequence models should easily predict, while the seen `cpu_bound` workload contains less predictable data where long-running processes are unpredictably preempted. We also wrote `split_by_workload.py`, which splits test data into workload-specific sets for per-workload evaluation.

3.2 Data Preprocessing

In this stage, we convert the raw scheduler trace CSV files into structured datasets that can be used for sequence modeling. The information that we get from raw logs include context switch events like an event's timestamp, the incoming process ID (PID), the state of the previous task, CPU ID and the time elapsed since the previous switch and this data is then transformed into a consistent numerical format since raw categorical values or irrelevant event streams can't be directly processed by neural networks.

In our pre-processing pipeline, we carry out three key operations: vocabulary construction, encoding features into numerical indices and extracting fixed-length temporal windows from the event stream. After we convert our raw data into supervised learning samples, the model can then then observe these sequences of past scheduling events and predict the next process to be scheduled.

(1) **Vocabulary construction.** First, we construct vocabularies for the dataset's categorical variables. In our project, the models use PID and previous task state to scan the training data for collecting all the unique values for these fields. We assign each unique PID and state value a numerical index via our mappings (stored in a JSON file) so that the same encoding can be applied to both our training and testing sets. To handle cases where the test set has a process not present in the training set, an additional unknown (UNK) token is also included in the vocabulary so that the model can safely map any unseen PID it encounters to this UNK index without throwing an error.

(2) **Encoding.** After our vocabulary is ready, the raw dataset is transformed into a fully numerical representation suitable for a neural network input. The PID and previous state values are converted into their corresponding indices using the vocabulary. The CPU ID and time difference fields are treated as continuous features and are converted into floating point values. Our three feature groups are now **PID index**, **state index**, and **continuous features** [cpu, time difference]. During this step, we sort the events by timestep so that the scheduling events are processed in the correct chronological order. This guarantees the validity of our temporal evaluation of the scheduler behavior we get based on these extracted sequences.

(3) **Sliding window extraction.** We then use the sliding window approach to convert the continuous event stream into fixed length sequences after the dataset has been encoded. For each trace, the preprocessing script pulls 64 consecutive scheduling events which forms an input to the model which predicts the PID of the next event based on this sequence. So now we have a dataset that's representative of a supervised learning problem where the model learns to predict the next scheduled process based on the recent scheduling history. A stride of 4 is also used when moving our window across the dataset making sure we don't lose a lot of context while still generating an appropriate amount of training samples for our computational resources to introduce some variance.

The final processed dataset is then stored as compressed NPZ files containing all the sequences which is then loaded by our training pipeline to create Pytorch datasets and data loaders.

3.3 LSTM Model

The LSTM baseline model proposed in the original paper is based on the sequence modeling approach where it learns the temporal scheduling patterns from event sequences. Here, we used the preprocessed inputs and training setup on the LSTM mentioned earlier. 64 scheduler events go into the model where instead of using one hot encodings like the original work, each event is represented by concatenating embedded categorical features (PID and previous state) with the 2 continuous features (CPU and time difference). We had a three-layer stacked LSTM through which when this sequence was passed, recurrent hidden steps were updated at each time step which modeled our temporal dependencies. The model was allowed to progressively learn richer temporal representations by feeding the output sequence from one layer into the next. Once a full sequence is processed, we took the final time step's hidden representation as a summary of the recent scheduling history. This history was then passed through a linear layer to predict the next PID. This baseline provided a good reference point for the evaluation of how effective the convolutional sequence modeling was in our TCN architecture.

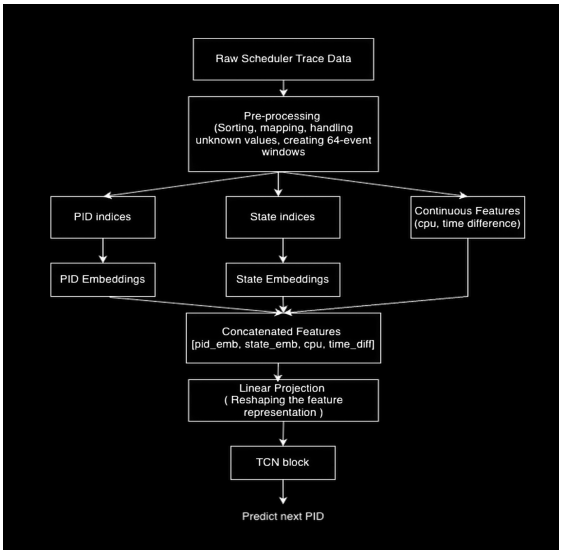

Figure 1: Model architecture diagram for the TCN model

3.4 TCN Model and Architecture

The Temporal Convolutional Network (TCN) is used to model temporal patterns in the scheduler event sequences. As mentioned, the model takes a sequence of 64 encoded events as input and gives us the predicted PID of the next scheduled process as output. See Figure 1 for the model's workflow.

After an event passes through the embedding layers that take care of the categorical variables, the PID indices are mapped to dense vectors of size 32 and state indices to vectors of size 8. These embeddings are then concatenated with the two continuous features (CPU and time difference) to form the full feature representation of each event, which is again projected into a higher dimensional representation using a linear projection layer to produce a 128-channel feature representation for each event in the sequence.

In the temporal modeling component. We have a stack of 6 residual blocks with two causal one-dimensional convolution layers present in each of them. Causal convolutions ensure that the representation at a timestep t is only based on the

current and past events so that the model can prevent from accessing any future information. With each successive residual block, the dilation factor increases exponentially to allow the model to capture long range temporal dependencies across the sequence without significantly increasing the computational cost. To stabilize training, the residual connections within each block adds the input of the block to its output. This also improves the gradient flow and allows more effective training of the deeper architectures.

Once the sequence has passed through all the residual blocks, the model extracts the final time step's corresponding representation and sends it to the linear classification layer which then produces logits over all possible PIDs. The model is trained using cross entropy loss to predict the correct next process.

To summarize the hyperparameters of the model architecture, we used 32 and 8 for the PID and state embeddings respectively and our combined feature vector is then projected into a 128 channel representation which is used throughout the convolutional network. We have 6 residual blocks, each using a kernel size of 3 and dropout of 0.1 for regularizations in the TCN. For training, the model uses AdamW for optimization with a learning rate of 0.001. A batch size of 128 is used to train the model.

4 Evaluation and Results

For the evaluation process, one of the metrics we chose was the top-1 accuracy, which measures the number of times that the top guess is being matched. We chose this over standard accuracy because in scheduling, a singular decision needs to be made. Top-1 accuracy was evaluated on both the seen and the unseen data in order to see whether performance sustains in unseen workloads. Another critical metric we evaluated is inference latency, which computes the time that it takes for a prediction to be made for the scheduler. One of the main concerns of the baseline was the fact that the prediction time was too long for true use in an operating system. Therefore, we wanted to understand whether TCN was faster in its inferences to judge its efficiency for kernel integration.

4.1 Prediction Top-1 Accuracy

Figure 2 represents the results of the top-1 accuracy of the LSTM baseline, as well our newly implemented TCN Model. These results demonstrate that the LSTM Model performed very poorly with seen data, and the TCN attained over 90% prediction accuracy in both the seen and unseen data.

With these results, there is a massive discrepancy in the correct prediction rate with TCN making massive improvements in that regard. We believe that the use of the convolution layers in the TCN model, smoother gradients are likely produced which helps in the training process. With RNNs, exploding and vanishing gradients are a pitfall in the training process.

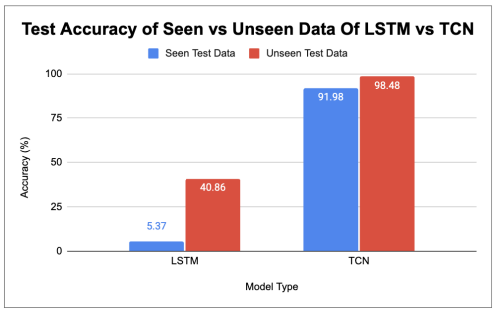

Figure 2: Top-1 Accuracy for TCN vs LSTM on the test dataset (seen and unseen data).

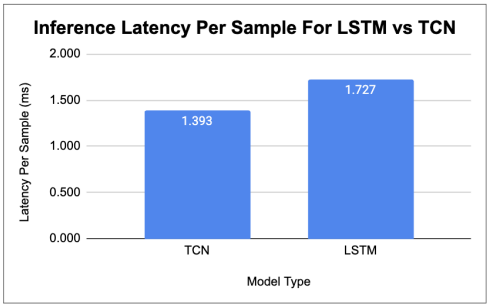

Figure 3: Inference latency per sample (ms) for LSTM baseline vs. TCN.

Although the LSTM solves the gradient problem to an extent, this challenge is less prevalent in a TCN due to its different backpropagation path [1]. Additionally, with its ability to capture longer range dependencies, this allows the model to be able to pick up longer scheduling sequences and patterns that repeat itself throughout the traces.

Additionally, another key takeaway from these results is that the **unseen accuracy was higher than the seen accuracy**, which was unexpected for us. However, conducting a trace analysis showed us that `sysbench_threads`, our unseen test, generates very repetitive scheduling patterns even with heavy background system noise. It's designed so that threads cycle through the scheduler in a regular rotation. Therefore, despite the fact the model did not see this workload during training, it was less difficult to predict the next decision with high confidence. Hence, LSTM, which performed poorly in predicting, generated a near 41% unseen test accuracy due to the ability to capture some of the repetitive pattern structure.

4.2 Inference Latency

Even though our TCN model's prediction accuracy strongly outperformed the LSTM model, we wanted to keep in mind the overhead it takes to make a prediction. This was a weakness in the original LSTM model. Figure 3 shows the inference latency per sample, which represents the average latency per prediction.

Workload	Accuracy (%)	Number of Samples
cpu_bound_4p	45.13	16,658
cpu_bound_8p	43.66	8,961
hackbench_pipe_large	97.15	504,252
hackbench_socket_large	94.89	214,731
io_mixed	53.63	12,369
sysbench_cpu_8t	50.03	19,636
sysbench_memory_4t	44.61	15,762

Table 2: Table showcasing the top-1 accuracies of TCN across the seen workloads, along with the number of samples for each workload.

These results show that compared to LSTM, which had an average inference latency of 1.727 ms, TCN had a lower latency of 1.393 ms. This is a **19% improvement** from the LSTM model. With TCNs, one of their biggest strengths is the fact that they are able to capture temporal dependencies that are farther in range, as long as be able to enforce more parallelism in its sequence processing. With the LSTM, which is a type of advanced Recurrent Neural Network (RNN), they are processed more sequentially which creates an overhead. Although this improvement still probably isn't feasible enough to be used in a real world operating system, it is a step in the right direction that there are better deep learning options than LSTMs that can predict scheduling patterns.

4.3 Generalization Breakdown by Workload

We also wanted to investigate further into the TCN's accuracy and evaluate how the model generalized on specific workloads. `sysbench_threads` is our unseen workload, where the accuracy was 98.48%. However, we wanted to observe the performance across seen workloads for evaluating generalization. Table 2 shows TCN top-1 accuracy broken down by individual workload type. Here, we can understand which workloads are more predictable and where the model struggles.

The `hackbench` workloads performed the best, which are the pipe/socket message passing workloads. They also were a very large majority of the generated trace, as shown in Table 2. The main reason is because our trace collection script only logs `sched_switch` events as explained in Sect 3.1. The purpose of the hackbench workload is to cause lots of context switches and the other workloads end up staying on the CPU for a longer period of time, which explains the discrepancy of data. Additionally, hackbench generated patterns that were more repetitive similar to `sysbench_threads`, so the model did a better job at making predictions. Many of the other workloads had more unpredictability and variance in the patterns which explains the model performing worse in those use cases.

5 Discussion

There are many differences that distinguish our work from the original KernalOracle paper and a key fundamental difference is *task formulation*. KernelOracle treats scheduling prediction as multivariate regression, which outputs a continuous vector whose closest task name is inferred after the fact. Our TCN treats it as discrete multi-class classification over a vocabulary of known processes, with cross-entropy loss directly penalizing wrong predictions. Treating predictive scheduling as a classification task rather than a regression task is an advantage because the focus is on inferring the correct PID, which is categorical and concrete.

Another advantage with our work is the fact that we were able to generate an *expanded dataset* for training the models. In the original paper, LSTM trained on ~17,000 events from one single workload. With our contributions, we were able to train the TCN on 1.8M sliding windows from 12 diverse workloads. The performance gap is therefore partly architectural and partly due to training data quality. We were able to modify the LSTM to account more for classification rather than regression and used our own dataset, but the performance discrepancy between the two models still existed.

Even though we achieved our main goals of implementing a machine learning model that can improve prediction accuracy and inference latency, limitations still exist. These weaknesses could be areas to address on in the future to improve our performance in the different metrics.

Inference latency still needs to improve. Despite our improvements, an inference latency of 1.393 ms remains far too slow for kernel integration. A typical CFS scheduling decision takes under 10 microseconds, so our model would still be 2–3 orders of magnitude slower due to adding milliseconds of latency to every context switch.

Add additional features. The TCN model currently uses only PID, process state, CPU core, and inter-arrival time. The `prev_comm` field and `sched_wakeup` events are captured by our pipeline but not yet used. It is possible that both of those data fields may improve prediction, because the task that wakes a sleeping process is often the one scheduled next.

6 Conclusion

We built upon KernelOracle's initial demonstration that Linux scheduling is learnable using machine learning. Here, we were able to addressing its two main limitations in the original work: restricted training data and lack of latency measurement. Our ftrace-based pipeline was able to capture traces across 12 diverse workloads, and our TCN achieves 92% top-1 accuracy on seen workloads and 98.5% on unseen workloads after five epochs, suggesting the scheduler's behavior is far more predictable than the baseline implied. Our inference latency also provided an improvement of close to 20%. While the reduced latency of TCN was not low enough to necessarily close the gap for kernel integration, the improvements in speed along with top-1 accuracy and data generation can be a benchmark for future work.

References

[1] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. 2018. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv:1803.01271 [cs.LG] https://arxiv.org/abs/1803.01271

[2] Ali R. Butt, Chris Gniady, and Y. Charlie Hu. 2005. The performance impact of kernel prefetching on buffer cache replacement algorithms. In *Proceedings of the 2005 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems* (Banff, Alberta, Canada) *(SIGMETRICS '05)*. Association for Computing Machinery, New York, NY, USA, 157–168. doi:10.1145/1064212.1064231

[3] Google. 2017. schedviz: Visualizing Linux Scheduler Behavior. https://github.com/google/schedviz GitHub Repository.

[4] Sampanna Yashwant Kahu. 2025. KernelOracle. https://github.com/SampannaKahu/KernelOracle GitHub Repository.

[5] Sampanna Yashwant Kahu. 2025. KernelOracle: Predicting the Linux Scheduler's Next Move with Deep Learning. arXiv:2505.15213 [cs.LG] https://arxiv.org/abs/2505.15213

[6] Linux Kernel Documentation. 2024. CFS Scheduler Design. https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html

Appendix A Other Measurements

From the inference latency per sample, we also were able to obtain the forward latency per batch for each model. This is measured by multiplying the inference latency by the batch size, which was fixed at 128 for both the LSTM and TCN. Results are shown here in Figure 4. As expected, the same trends that we received with our inference latency stayed the same.

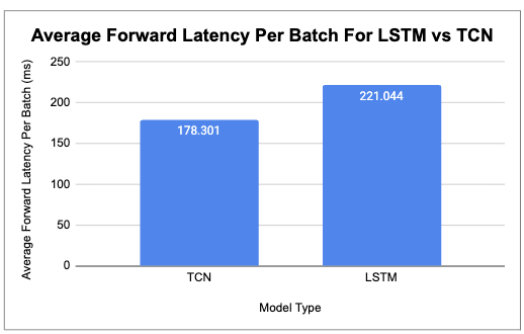

Figure 4: Forward latency per batch (ms) for LSTM baseline vs. TCN.

Additionally, this batch size is an important number for our metrics collection because the batch size is used to compute the throughput consumption of each of the models. The formula for throughput is given in Equation 1:

$$Throughput_{model} = \frac{BatchSize_{model}}{InferenceLatency_{model}} \times 1000 \quad (1)$$

We multiply by 1000 to make the units samples/sec, rather than samples/ms. Figure 5 demonstrates the throughput comparison for LSTM and TCN. Based on the results, TCN achieves a 24% improvement in its throughput, going from 579.07 samples/sec to 717.89 samples/sec.

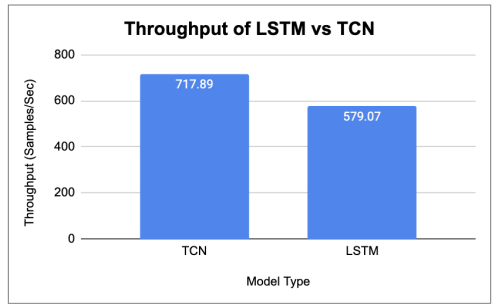

Figure 5: Throughput (in samples per second) of LSTM vs. TCN.

Appendix B Possible Applications

In spite of these limitations, our results demonstrate that Linux scheduling behavior is highly predictable from recent execution history: 91–98% top-1 accuracy after one epoch, across diverse workloads, indicates the scheduler follows learnable patterns. One application is prefetching [2]: knowing the next process allows the kernel to prefetch its data or instructions into cache during the context switch. A second is adaptive time slicing: if the model predicts the same process for several consecutive events, the kernel could extend its time slice to reduce context-switch overhead.